Name: Abhishek Kamlesh Vishwakarma
Class: D20C **Roll.No:** 35

Experiment No.1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. Hash Generation using SHA-256:
   - Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

2. Target Hash Generation with Nonce:
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

3. Proof-of-Work Puzzle Solving:
   - Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

4. Merkle Tree Construction:
   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

Program #1: Python program that uses the hashlib library to create the hash of a given string:

XXXXXXX #error statement; 3 Error

```
import hashlib                                                  #error statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()                       #error statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)                                      #error statement
```

```
Enter a string: abhishek
Hash: 42b5bf6a3fa51849bc05c7050141a81591133b5f470cdef4ff6628e1222ff0a1
```

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce                                  # Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)                                      # Error
```

```
Enter a string: jamming
Enter the nonce: singing
Hash Code: f0cf490ff9d3b05b2a0d4cb74b793fbab3fe243b3fa41c347e19e152dc3be164
```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```python
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)                    #Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):                          #Error
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "Abhishek"
num_zeros = 5

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)                              #Error
```

```
Hash: 0000035a1dafbe35a50c22885188e74f12ff953df1305a12779ef08a11f80ee8
Input String: Abhishek
Leading Zeros: 5
Expected Nonce: 445837
```

Program 4: Generating Merkle Tree for given set of Transactions

```python
import hashlib

def build_merkle_tree(transactions):                          # Error
    if len(transactions) == 0:
        return None

    if len(transactions) == 1:
        return transactions[0]

    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i+1]
            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest()
            new_transactions.append(hash_combined)                # Error

        transactions = new_transactions

    return transactions[0]

# Example usage
transactions = ['Alice->Bob:$200', 'Bob->Dave:$500', 'Dave->Eve:$100', 'Eve->Alice:$300', 'Roo->Bob:$50']

merkle_root = build_merkle_tree(transactions)
print("Merkle Root:", merkle_root)                            # Error
```

```
Merkle Root: 9cda8883bd77297aae6c50ae581a6d7c0db8ff297be36665a0fc8d91829af154
```

All the intermediate hash during the construction of Merkle Tree Root Hash:

```python
import hashlib

# Function to calculate SHA-256 hash
def calculate_hash(data):
    return hashlib.sha256(data.encode('utf-8')).hexdigest()

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    # Step 1: Hash all transactions
    print("Initial Transaction Hashes:")
    hashed_transactions = []

    for i, tx in enumerate(transactions):
        tx_hash = calculate_hash(tx)
        hashed_transactions.append(tx_hash)
        print(f"T{i+1}: {tx}")
        print(f"Hash : {tx_hash}\n")

    level = 1

    # Step 2: Construct Merkle Tree
    while len(hashed_transactions) > 1:
        print(f"Level {level} Hashes:")

        # Duplicate last hash if number of hashes is odd
        if len(hashed_transactions) % 2 != 0:
            hashed_transactions.append(hashed_transactions[-1])

        new_hashes = []

        for i in range(0, len(hashed_transactions), 2):
            combined = hashed_transactions[i] + hashed_transactions[i + 1]
            parent_hash = calculate_hash(combined)

            print(f"Hash {i//2 + 1}")
            print(f" Left : {hashed_transactions[i]}")
            print(f" Right: {hashed_transactions[i + 1]}")
            print(f" Parent Hash: {parent_hash}\n")

            new_hashes.append(parent_hash)

        hashed_transactions = new_hashes
        level += 1

    return hashed_transactions[0]

# Sample Transactions
transactions = [
    "Alice -> Bob : $200",      # T1
    "Bob -> Dave : $500",       # T2
    "Dave -> Eve : $100",       # T3
    "Eve -> Alice : $300",      # T4
    "Roo -> Bob : $50"          # T5
]

# Generate Merkle Root
merkle_root = build_merkle_tree(transactions)

print("Final Merkle Root:")
print(merkle_root)
```

```
Initial Transaction Hashes:
T1: Alice -> Bob : $200
Hash : f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760

T2: Bob -> Dave : $500
Hash : 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46

T3: Dave -> Eve : $100
Hash : 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da

T4: Eve -> Alice : $300
Hash : cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a

T5: Roo -> Bob : $50
Hash : cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
```

```
Level 1 Hashes:
Hash 1
 Left : f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
 Right: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
 Parent Hash: 471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9

Hash 2
 Left : 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
 Right: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
 Parent Hash: a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7

Hash 3
 Left : cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
 Right: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
 Parent Hash: 314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Level 2 Hashes:
Hash 1
 Left : 471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9
 Right: a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
 Parent Hash: ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee

Hash 2
 Left : 314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
 Right: 314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
 Parent Hash: 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

Level 3 Hashes:
Hash 1
 Left : ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
 Right: 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45
 Parent Hash: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Final Merkle Root:
96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0
```

Conclusion:
This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target** Hashing allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work** Puzzle Solving shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.